Física Computacional para Data Science

Tarea 03

PDEs Parabólicas y Elípticas: Gestión Térmica y Ecuación de Poisson

Fecha de entrega: 16/10/26 - 22:00 - Defensa presencial: 20/10/26 - 16:30

Profesor: Pablo Benítez Llambay | Universidad Adolfo Ibáñez

- **Formato de entrega:** Un único Jupyter Notebook (`.ipynb` y `.html`) con celdas ejecutadas, gráficos reproducibles y análisis conceptuales detallados. El encabezado del notebook debe mostrar claramente el nombre del autor. **No se aceptarán informes sin nombre.**
- **Objetivo principal:** Implementar algoritmos de diferencias finitas para resolver ecuaciones parabólicas (difusión con fuente) y elípticas (Poisson), evaluando criterios de estabilidad, optimización por sobrerrelajación sucesiva (SOR), tasas de convergencia espacial y costo computacional.
- **Resultados esperados:** Contrastar cuantitativamente la simulación transitoria frente a la resolución directa del estado estacionario. Demostrar dominio de la formulación numérica implementada.
- **Uso de IA:** El uso de herramientas de IA está permitido exclusivamente como soporte técnico de consulta. Todo el código, las deducciones matemáticas, los gráficos y las discusiones conceptuales deben ser comprendidos a cabalidad por el estudiante. La entrega del trabajo implica la total autoría y dominio de la solución presentada. Cada estudiante deberá ser capaz de responder preguntas de forma individual durante la sesión de clase dedicada a la presentación del trabajo. En particular, deberá ser capaz de explicar y justificar minuciosamente cualquier línea de código, elección de estructura de datos, algoritmo numérico o razonamiento físico presente en su entrega. La incapacidad de explicar una implementación o la justificación de resultados erróneos atribuibles a una copia acrítica de modelos de IA podrá resultar en la anulación de la totalidad de la evaluación.

Gestión térmica de un procesador multinúcleo

Durante la ejecución de cómputo de alto rendimiento, la disipación térmica en la CPU puede alcanzar límites críticos. Modelaremos el chip como una placa delgada cuadrada de dimensiones normalizadas $[0, L] \times [0, L]$ con $L = 1.0$, sujeta a una difusividad térmica constante $\alpha = 1.0$.

La evolución temporal del campo de temperatura $T(x, y, t)$ está gobernada por la ecuación parabólica de calor con generación interna S:

$$\frac{\partial T}{\partial t} = \alpha \left(\frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2} \right) + S(x, y)\,, \tag{1}$$

donde $S(x, y)$ cuantifica el calor generado por unidad de volumen.

Condiciones Físicas

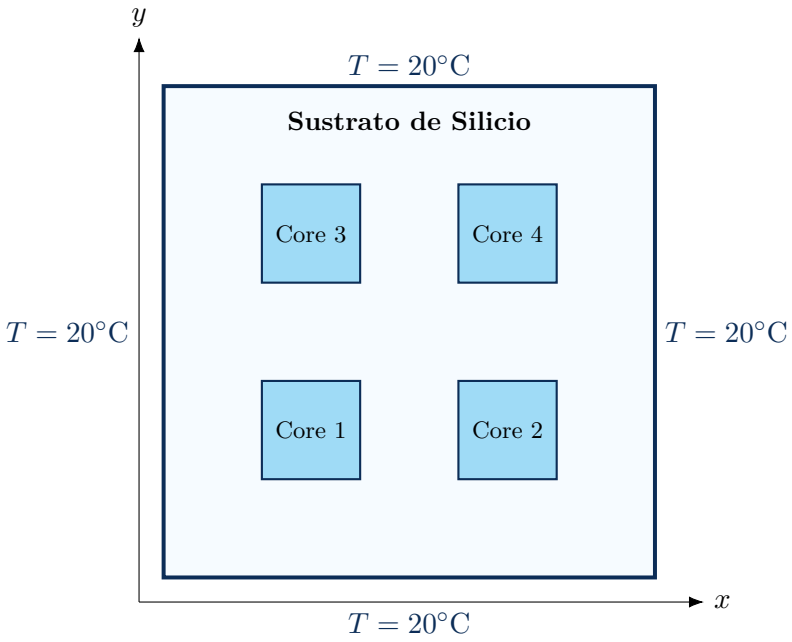
- **Condición inicial:** La CPU se enciende desde una temperatura ambiente uniforme: $T(x, y, 0) = T_{\text{ext}} = 20.0°\text{C}$.
- **Condición de borde:** La periferia del procesador está en contacto perfecto con un bloque disipador refrigerado:

$$T(0, y, t) = T(L, y, t) = T(x, 0, t) = T(x, L, t) = T_{\text{ext}} = 20.0°\text{C}.$$

- La CPU dispone de cuatro núcleos simétricos de tamaño 0.2×0.2 ubicados en:

$$[x_{1,k}, x_{2,k}] \times [y_{1,k}, y_{2,k}], \quad k \in \{1, 2, 3, 4\}$$

definidos por los cuadrantes: $[0.2, 0.4] \times [0.2, 0.4]$, $[0.6, 0.8] \times [0.2, 0.4]$, $[0.2, 0.4] \times [0.6, 0.8]$ y $[0.6, 0.8] \times [0.6, 0.8]$. Dentro de cada núcleo la disipación es intensa y uniforme, $S(x, y) = S_0 = 2000.0$ y fuera de los núcleos es $S(x, y) = 0.0$.



Dinámica Transitoria

1. Discretice el dominio espacial con $N_x = N_y = 101$ puntos. Formule e implemente el esquema explícito centrado en espacio y adelantado en tiempo (FTCS 2D) para la Ec. (1).
2. Integre el sistema desde $t = 0.0$ hasta un tiempo físico total $t_{\text{final}} = 0.5$. Utilice el límite de estabilidad de Von Neumann en dos dimensiones ($\Delta t \leq \dfrac{\Delta x^2}{4\alpha}$). Elija un factor de seguridad adecuado.
3. Registre la evolución de la temperatura central del *Core 1* en función del tiempo físico t. Si la temperatura crítica se define en $T_c = 85.0°\text{C}$, determine con precisión en qué instante la CPU alcanza dicha temperatura crítica.
4. Mida y reporte:
   - El número total de pasos temporales requeridos para alcanzar t_{final}.
   - El tiempo de CPU utilizado para resolver el problema.
   - Cuantifique el error cometido en el estado obtenido como:

$$R = \max_{i,j} \left| \frac{T_{i,j}^{n+1} - T_{i,j}^{n}}{\Delta t} \right|$$

   ¿Se puede afirmar que el sistema alcanzó el estado estacionario en términos prácticos?

Estado Estacionario

En el límite $t \to \infty$, la derivada temporal se anula ($\partial_t T = 0$), y la distribución térmica converge a la solución de la ecuación de Poisson:

$$\nabla^2 T = -\frac{S(x,y)}{\alpha} \tag{2}$$

Nota sobre rendimiento y tiempos de cómputo

A diferencia de los esquemas matriciales vectorizables (como FTCS o Jacobi), los métodos de relajación como **Gauss-Seidel** y **SOR** imponen dependencias secuenciales.
En Python, implementar estos algoritmos mediante `for` anidados sobre una grilla 2D puede requerir tiempos de ejecución muy grandes (como se experimentó en la clase de ecuaciones elípticas).
Para optimizar el cálculo, se sugiere investigar e implementar alguna de las siguientes alternativas:

- **Compilación JIT con Numba (`@njit`):** Permite acelerar `for` nativos de Python a velocidad cercana a C/Fortran de forma sencilla.
- **Esquema tablero de ajedrez:** Desacopla la dependencia de datos dividiendo la grilla en celdas pares e impares, permitiendo vectorización con NumPy.
- **Cython o módulos en C:** Para quienes deseen compilar la rutina de actualización externa.

Recomendación práctica: Valide sus algoritmos en una grilla reducida (e.g., 21×21 o 51×51) antes de ejecutar las resoluciones de producción.

1. Implemente un método de Sobrerrelajación Sucesiva (SOR) para la Ec. (2) con la fuente $S_{i,j}$ discretizada en la misma malla de 101×101. Utilice un criterio de parada:

$$\max_{i,j} |T_{i,j}^{k+1} - T_{i,j}^{k}| < 10^{-5}$$

2. Realice un barrido del factor de relajación $\omega \in [1.0, 1.98]$ con un espaciado fino. Grafique el número de iteraciones necesarias para converger en función de ω. Identifique $\omega_{\text{óptimo}}$.

3. Compare cuantitativamente los métodos de Jacobi ($\omega = 1$ simultáneo), Gauss-Seidel ($\omega = 1$ secuencial), SOR con $\omega_{\text{óptimo}}$ y el esquema temporal FTCS de la Parte 1. Construya una tabla comparativa que resuma:

Método	**Iteraciones/Pasos**	**Tiempo CPU [s]**	$T_{\max}$ [°C]	**Residuo** R
FTCS (Parabólico)				
Jacobi				
Gauss-Seidel				
SOR ($\omega_{\text{óptimo}}$)				

Discuta la eficiencia de abordar el problema elíptico directamente frente a integrar el transitorio físico.